

完全面试手册

叶俊聪（高手）

251732351 - 软件工程（大数据与人工智能学院）

Github: [Ayeez757 \(阿叶Ayeez\)](#)

Bio: Java 为主，对架构 / 性能 / 运维感兴趣，探索中...；

技术栈: Java、SpringBoot、MyBatis、MySQL、Redis、RabbitMQ、MQTT、Netty、WebSocket、Docker、K8s、HPA、Prometheus、Grafana、ELK、Nginx、CI/CD、Linux、JMeter 等

独立项目: [Ayeez757/AyeezBlog: 阿叶Ayeez酷炫的博客! AyeezBlog 是一个开源的博客系统, 采用Springboot+vue3的前后端分离架构设计。拥有前台和管理端双端, 支持 Markdown 文章、评论互动、分类标签、归档与友链等博客常用能力。整体风格酷炫, 技术栈覆盖前端工程化、后端安全与数据访问、缓存、容器化部署和自动化流程, 适用于个人博客、技术社区与中小型内容站点的搭建和二次开发。](#)

第二轮考核

[Ayeez757/accountingSystem: 爪哇部落后端组二轮考核, 基于springboot的记账系统后端](#)

技术栈: Java + Springboot + Maven + MySQL

评语: 同学你好, 整体项目架构体现了你良好的 MVC 设计意识。同时对认证流程已经有了系统性的理解, 整体完成度较高, 数据库表设计也较为完整。美中不足的是你管理员鉴权目前是在每个 Controller 方法内手动解析 Token 判断 role, 建议统一在拦截器或 Filter 中处理, 保证权限控制的一致性。总的来说项目完成度高, 技术选型成熟, 能主动实践加密、JWT、定时任务等后端核心知识, 基础扎实, 继续加油!

第三轮考核

<https://github.com/Ayeez757/AyeezBlog/tree/25-backendRecruit3-hotReload>

技术栈: java, springboot, mysql

AIGC 使用情况: 学习计划的制定; 与ai进行多轮对话帮助捋清JUC、JVM相关概念, 热重载方案设计的完善; 并发安全与失败回退策略梳理; 具体代码实现, 使用大量vibecoding; 测试脚本的编写; 验证步骤与交付文档初步编写。

Summary / 概述

本项目在 AyeezBlog 博客系统中实现了热重载子系统, 技术栈为 Java 17 + Spring Boot 3.2.0 + MySQL, 采用 Maven 多模块结构 (blog-common / blog-pojo / blog-server / blog-plugin-demo)。

项目完成了 Level 1（配置文件热重载）和 Level 2（业务逻辑热重载）两个等级：

- **Level 1:** `RuntimeConfigLoader` 加载 YAML 配置 → `RuntimeConfigManager` 以 `AtomicReference` 做快照原子替换 → `RuntimeConfigWatchService` 通过 Java NIO `WatchService` 监听文件变动（800ms 防抖）。配置变更时旧快照保持可用, 新配置失败时自动回退。
- **Level 2:** 定义 `RulePlugin<I, O>` 泛型接口（id/init/execute/dispose 生命周期），`PageSizeRuleEngineService` 管理插件切换（先初始化新插件 → 原子替换引用 → 释放旧插件 + 平台兜底清理 + 关闭旧 `ClassLoader`）。支持外部 JAR 热加载（`URLClassLoader` + `ExternalPageSizePluginWatchService` 监听 `plugins/page-size/` 目录, 1200ms 防抖 + 250ms 文件稳定性检查）。

项目提供了一个 demo 控制器（8 个 REST 端点）用于演示热重载效果, 两个示例插件（无状态的 `DoublePageSizeRulePlugin` 和有副作用的 `StatefulSideEffectPageSizeRulePlugin`），以及 PowerShell/Bash 自动化测试脚本（300 轮 A/B 对照试验 + 资源泄漏检测）。实际业务集成点为 `PostServerImpl.listDetail()` 中的分页大小计算。

Rebuttal 后的补充评审

作者在收到初次反馈后提交了 Rebuttal, 并在 `hotReload-v2` 分支上进行了以下改进：

1. **Docker Compose 评审环境：** `deploy/review/docker-compose.yml` 提供 MySQL 8.4 + Spring Boot + Vue/Nginx 一键栈, 取消了七牛、DeepSeek、RabbitMQ 等可选依赖的硬性要求
2. **跨平台脚本：** 在保留 PowerShell 脚本的基础上, 增加了 Bash（*.sh）脚本, 支持 macOS / Linux / Git Bash
3. **插件开发文档：** `docs/分页规则插件_第三方SDK说明.md` 详细说明了 `RulePlugin` 接口契约、`plugin.properties` 格式、编译依赖与部署约定
4. **评审入口脚本：** `docker/review-entrypoint.sh` 自动将 demo JAR 同步到 `bundled-demo` 目录, 评审者只需复制到 `page-size` 即可触发 Level 2

Docker 实测结果 (WSL2 + Docker Compose):

- 三个容器均成功启动并通过健康检查 (MySQL / Backend / Frontend)
- 配置热重载验证通过: 修改 `runtime-config.yml` 的 `postPageSize` (5→8), `/post/runtime/config` 端点在 ~2 秒内自动反映变更, `/post/runtime/demo-cards` 端点返回的卡片数量同步变化
- 内置插件切换验证通过: `/post/runtime/switch-page-size-rule?pluginId=runtime-only-page-size-rule` 成功切换, 端点返回正确的 `lastSwitchSource: "manual"`
- 外部 JAR 热加载验证通过: 将 `blog-plugin-demo-0.0.1-SNAPSHOT.jar` 复制到 `plugins/page-size/`, `ExternalPageSizePluginWatchService` 自动检测并加载, `/post/runtime/page-size-rule` 显示 `currentPluginOrigin: "external"`, `currentExternalJarPath` 指向正确路径
- 有副作用插件的资源追踪可观察: `StatefulSideEffectPageSizeRulePlugin` 的心跳日志 (`side-effect-heartbeat.log`) 记录了实例 ID 和活跃实例数

发现的问题: `hotReload-v2` 分支存在编译错误——`PageSizeRuleEngineService.java` 第 55-62 行误合入了 `StatefulSideEffectPageSizeRulePlugin` 的 `releaseResources()` 方法, 引用了未定义的变量 (`released`、`ACTIVE_HEARTBEAT_TASKS`、`ACTIVE_PLUGIN_INSTANCES`、`instanceId`)。评审者需手动删除该方法块后方可编译通过。这可能是合并冲突解决时的疏忽。

Strengths / 主要优点

1. **热重载架构设计成熟:** 配置层采用快照语义 (`AtomicReference` 整体替换), 业务线程读取无锁; 插件层采用"先初始化 → 再替换 → 再释放"三步切换, 确保切换窗口期内始终有可用插件。失败回退机制完备——新插件 `init()` 失败时主动清理半初始化资源, 不影响当前服务。这在并发安全设计上是正确的做法。
2. **资源回收机制考虑周全:** `PluginContext` 使用 `ThreadLocal` 追踪初始化作用域 + `IdentityHashMap` 登记清理动作, `pluginCleanupEnabled` 配置项可开关平台回收。外部 `URLClassLoader` 在插件切换和销毁时均有 `closeClassLoaderQuietly()` 兜底。有副作用的 demo 插件通过 `registerCleanupAction()` 展示了清理链路的完整闭环。
3. **外部插件加载的工程细节到位:** `ExternalPageSizePluginWatchService` 不仅做了防抖 (1200ms), 还做了文件写入稳定性检查 (250ms 内文件大小不变才触发加载), 避免了 JAR 尚未写完就被加载的经典问题。`ExternalPluginDescriptorReader` 从 JAR 内读取 `plugin.properties` 做描述符校验。
4. **A/B 对照测试方法论正确:** 300 轮自动化热加载对照试验 (Group A: `pluginCleanupEnabled=false` vs Group B: `pluginCleanupEnabled=true`), 通过对比线程数增长、内存占用、心跳日志增长等指标验证资源回收的有效性。测试脚本 (`run-level2-auto-watch.ps1`、`check-level2-leak-signals.ps1`) 可复现, 且已补充 Bash 版本。
5. **切换历史可审计:** `PageSizePluginSwitchRecord` 记录每次切换的时间戳、来源 (`startup/runtime-config/manual`)、来源类型 (`built-in/external`)、插件 ID、外部 JAR 路径、成功/失败状态和消息, 最多保留 50 条。便于评审和自测时确认切换行为。
6. **代码文档完备:** 所有公开类和方法均有 Javadoc, 解释了设计意图而非简单复述方法签名。例如 `PageSizeRuleEngineService.switchPlugin()` 的 Javadoc 明确说明了"先初始化新插件, 再替换当前引用, 最后释放旧插件"的顺序及其原因。
7. **局限性诚实透明:** 交付文档明确声明"单插件清理可行, 跨插件副作用清理不完整", 未过度包装实现范围。作者在 Rebuttal 中坦承了选题决策失误、AIGC 比例 (~70%)、以及 IO 会话场景的能力边界。
8. **持续稳定的开发投入:** Git 历史显示 354 次提交 (2026-02-19 至 2026-05-01), 跨越 2.5 个月, 单人开发。约 40% 的提交遵循 Conventional Commits 格式 (`feat:`、`fix:`、`refactor:`、`ci:`), 其余为描述性中文提交。有专门的功能分支 (`25-backendRecruit3-hotReload`)、发布标签 (`release-v1.0.0` 至 `release-v1.16.0`) 和实验分支, 体现了持续的工程实践而非临阵突击。
9. **Rebuttal 改进有效:** Docker Compose 评审环境大幅降低了复现成本; 跨平台脚本解决了 PowerShell 专属依赖问题; 插件开发文档使评审者可以理解插件契约。entrypoint 脚本的设计 (镜像内 JAR → `bundled-demo` → 按需复制到 `page-size`) 清晰地分隔了 Level 1 和 Level 2 的验证路径。

Weaknesses / 主要缺点

1 运行与测试障碍

1. **热重载功能无法脱离博客系统独立运行:** 虽然 Docker Compose 简化了部署, 但热重载子系统仍然嵌入在完整的博客项目中。`blog-server` 模块的 Spring 上下文初始化仍会尝试连接 MySQL、扫描 RabbitMQ 配置等 (review profile 下这些被跳过但依赖仍在)。评审者无法仅启动热重载子系统进行验证。
2. **无单元测试:** 热重载核心逻辑 (`RuntimeConfigManager.reload()` 的原子替换与失败回退、`PageSizeRuleEngineService.switchPlugin()` 的并发安全、`PluginContext` 的清理登记与执行、`ExternalPageSizePluginLoader` 的类加载验证) 均无 JUnit 测试覆盖。`testLevel2.java` 整个文件被注释掉, 仅为设计草稿而非可运行测试。测试完全依赖 PowerShell/Bash 集成脚本, 但这要求整个应用先跑起来。

3. **hotReload-v2 分支存在编译错误**: `PageSizeRuleEngineService.java` 第 55-62 行误合入了 `StatefulSideEffectPageSizeRulePlugin` 的 `releaseResources()` 方法, 引用了未定义的变量。评审者需手动删除该方法块后方可编译通过。这在 Docker 构建时会导致失败, 降低了评审体验。
4. **README 结构不利于评审**: 后端子目录 `AyeezBlog-Backend/` 内无 README, 主 README 位于上层 `AyeezBlog/` 目录, 且为博客系统整体说明而非热重载专项。评审者需自行在 `docs/` 目录下找到交付文档。考核文档要求 README 包含“热重载的触发方式与验证步骤”, 但这些信息分散在多个文件中。

2 代码与设计问题

5. **热重载对博客业务的实际影响存疑**: 热重载系统的唯一业务集成点是 `PostServerImpl.listDetail()` 中的分页大小计算 (`pageSizeRuleOutput.getEffectivePageSize()`)。这意味着切换插件后, 唯一可观察到的变化是“每页显示多少篇文章”——博客的实际渲染内容、文章解析逻辑、模板渲染策略等核心业务完全不受热重载影响。作为一个博客系统, 热重载更自然的切入点应该是: 文章渲染模板的热切换 (Markdown → HTML 转换规则)、评论过滤策略的热替换、或缓存策略的动态调整。当前实现将热重载局限在一个几乎不影响用户体验的分页参数上, 削弱了“业务逻辑热重载”的说服力。作者在 Rebuttal 中坦承这是能力边界所致, 未能在反馈周期内补上更核心的接入。
6. **选择复杂的博客项目作为基础, 技术判断存疑**: AyeezBlog 是一个功能完整的博客系统, 依赖 MySQL、RabbitMQ、七牛云、DeepSeek AI 等多个外部服务。据了解, 该项目此前曾因鉴权逻辑缺陷被攻击者通过查阅源码注入后台管理界面。在此背景下, 作者仍选择将热重载子系统嵌入如此复杂的项目中, 而非构建一个轻量级的独立服务, 这带来了两个问题: 一是大幅提高了评审门槛 (评审者需要搭建完整博客环境); 二是热重载子系统被博客系统的复杂性所淹没, 评审者难以聚焦于热重载本身的设计质量。作者在 Rebuttal 中承认这是决策失误——对现有代码更熟悉, 且手上没有另一套现成的服务。
7. **demo 插件未触及热重载核心难点, 存在测试不全的质疑**: 热重载真正的难点在于 IO 会话管理——文件 IO 句柄的跨版本生命周期控制、数据库连接/迁移的无缝切换、正在处理中的请求如何优雅过渡到新逻辑。然而两个 demo 插件均未触及这些场景: `DoublePageSizeRulePlugin` 仅将配置值乘 2 (纯内存运算), `StatefulSideEffectPageSizeRulePlugin` 的“副作用”是一个写心跳日志的 `ScheduledExecutorService`——本质仍是配置值变换 + 一个可控的定时器。没有涉及数据库查询规则切换、文件解析策略替换、或正在处理中的 IO 请求如何在插件切换时不丢失状态等真实场景。作者在 Rebuttal 中坦承这是能力不足所致, “尝试过拉了坨大的, 但是全部 reset 了”。
8. **plugin.properties 配置遗漏**: `blog-plugin-demo` 模块下 `plugin.properties` 和 `plugin-stateful.properties` 内容完全相同, 均指向 `StatefulSideEffectPageSizeRulePlugin`。 `DoublePageSizeRulePlugin` 无法通过 `ExternalPluginDescriptorReader` 被正确发现 (虽然手动加载仍可工作), 这是一个明确的疏忽。
9. **resolvePageSize() 中外部插件与配置热重载的交互语义不清晰**: 当 `currentPluginOrigin == "external"` 时, `resolvePageSize()` 跳过配置检查, 外部插件持续生效。但此时若修改 `runtime-config.yml` 中的 `pageSizeRulePluginId`, 配置热重载会触发 `RuntimeConfigManager.reload()` 更新快照, 却不会影响已接管的外部插件。作者在 Rebuttal 中解释这是有意为之——避免运维通过目录投放外置 JAR 后, 仅因编辑配置文件就静默退回内置实现。这一设计决策合理, 但应在代码注释或交付文档中明确说明。
10. **RuntimeConfigDemoController 缺少鉴权**: 所有热重载管理端点 (手动切换插件、加载外部 JAR、触发配置重载) 均无任何鉴权或权限控制。在生产环境中, 这些端点可被任意调用, 存在安全风险。

3 文档与工程细节

11. **testLevel2.java 不应出现在交付代码中**: 整个文件被注释掉, 包含伪代码设计草稿。交付代码中的注释掉的代码应被清理或转化为正式测试。
12. **StrictModePageSizeRulePlugin 命名不一致**: `RuntimeConfig` 中有 `strictModeEnabled` 字段, `DefaultPageSizeRulePlugin` 根据此字段切换行为, 但没有名为 `StrictModePageSizeRulePlugin` 的独立插件。内置插件 ID 为 `default-page-size-rule` 和 `runtime-only-page-size-rule`, 前者的行为取决于 `strictMode` 开关——这种隐式分支增加了理解成本。

Questions for Authors / 给作者的问题

1. 当前热重载系统的唯一业务集成点是分页大小计算 (`PostServerImpl.listDetail()` 中的 `getEffectivePageSize()`), 切换插件后唯一可观察到的变化是“每页显示多少篇文章”。博客的实际渲染内容、文章解析逻辑等核心业务不受影响。你是否考虑过将热重载应用于更核心的业务场景 (如文章渲染模板热切换、评论过滤策略热替换)? 选择仅控制分页大小是技术限制还是设计取舍? **已由作者 Rebuttal 回答**: 作者坦承这是能力边界所致, 未能在反馈周期内补上更核心的接入。
2. AyeezBlog 是一个依赖 MySQL、RabbitMQ、七牛云、DeepSeek AI 等多个外部服务的动态博客系统, 其前身并未实现热重载功能。即使如此, 你为什么依旧选择将热重载嵌入如此复杂的项目中, 而非构建一个轻量级的独立服务? 你是否评估过这个选题策略对评审门槛的影响? **已由作者 Rebuttal 回答**: 作者承认这是决策失误——对现有代码更熟悉, 且手上没有另一套现成的服务。
3. 热重载的核心难点通常在于 IO 会话管理——例如文件 IO 句柄的跨版本生命周期、数据库连接/迁移的无缝切换、正在处理中的请求如何在插件切换时保持一致。当前两个 demo 插件仅做纯内存运算 (配置值乘 2) 和写心跳日志, 未涉及这些场景。这是时间限制下的取舍, 还是你认为这些场景不属于热重载的考察范围? 如果有更多时间, 你会设计什么样的 demo 插件来验证 IO 会话控制能力? **已由作者 Rebuttal 回答**: 作者坦承这是能力不足所致, “尝试过拉了坨大的, 但是全部 reset 了”。

4. **(保留)** `resolvePageSize()` 中, 外部插件接管后会跳过 `runtime-config.yml` 的配置检查。如果此时管理员修改了配置文件中的 `pageSizeRulePluginId`, 配置热重载会更新快照但不会影响外部插件的行为。你认为这是合理的"外部插件显式优先"语义, 还是应该在配置变更时自动回退到内置插件? **作者已回答**: 这是有意为之的设计, 避免运维通过目录投放外置 JAR 后仅因编辑配置文件就静默退回内置实现。建议在代码注释中明确说明这一设计决策。
5. **(保留)** `PluginContext` 使用 `IdentityHashMap` 按插件实例登记清理动作。如果同一个插件类被加载两次 (两个不同的实例), 清理动作是隔离的。但如果插件内部使用了 `static` 资源 (如 `StatefulSideEffectPageSizeRulePlugin` 中的 `static Set<Thread> heartbeatTasks`), 旧实例的 `static` 状态在 `ClassLoader` 未被完全卸载时仍会残留。你在设计中是否考虑过 `ClassLoader` 泄漏导致的 `static` 状态残留问题? **作者已回答**: 承认之前未考虑此问题, 分析了不同 `ClassLoader` 加载的类拥有独立的 `static` 状态, 但无法完全解决 `ClassLoader` 泄漏问题。
6. **(保留)** 交付文档中提到"跨插件副作用清理不完整", 能否具体说明哪些场景下清理会失败? 你认为理想的解决方案是什么? **作者已回答**: 平台按实例登记的 `IdentityHashMap` 无法覆盖进程级共享资源 (线程池、系统属性、单例、`ThreadLocal`、文件、端口), 建议通过严格的插件开发规范来约束。
7. **(保留)** AIGC 使用情况中披露了作者使用了大量 VibeCoding 实现, 能否估计这类代码的比例? 你是否对生成的代码进行了充足的人工审查并且有信心对生成的代码负责? **作者已回答**: vibecoding 比例预估七成左右, 确保了对代码有审查, 但坦承"有信心对生成的代码负责"这一说法需要修正——考核效果不达标是实现方案本身的问题, 而非代码质量问题。

Overall Assessment / 总体评分

- **Core features:** 3.0/4.0 - Level 1 和 Level 2 在代码层面均有实现, 热重载架构设计正确 (快照原子替换、先初始化再切换、失败回退、`ClassLoader` 管理)。Docker 实体验证了配置热重载、内置插件切换、外部 JAR 热加载三条路径均可工作。扣分原因: 热重载唯一业务集成点仅影响分页大小, 未触及博客渲染等核心业务; 无单元测试验证核心逻辑的正确性; `hotReload-v2` 分支存在编译错误
- **Extra features:** 1.5/2.0 - 提供了 demo 控制器 (8 个 REST 端点)、两个示例插件 (无状态 + 有副作用)、Bash + PowerShell 自动化测试脚本 (300 轮 A/B 试验 + 泄漏检测)、切换历史审计、Docker Compose 评审环境、插件开发文档。较初次提交有明显改进。扣分原因是 demo 插件仅做纯内存运算 (配置值乘 2), 未涉及 IO 会话控制等热重载核心难点; `plugin.properties` 配置遗漏; 管理端点无鉴权
- **Documentation completeness:** 1.5/2.0 - 交付文档详尽 (516 行中文 + 英文版), 代码 Javadoc 完备, AIGC 使用披露透明, Rebuttal 坦诚回答了所有问题。插件开发文档 (分页规则插件_第三方 SDK 说明.md) 是有效的补充。扣分原因是 README 结构分散、热重载验证步骤未集中在一处、编译插件仍依赖主服务构建产物
- **Code style:** 1.5/2.0 - 多模块分层清晰, 命名规范符合 Java 惯例, 并发设计使用 `AtomicReference` + `synchronized` + `ThreadLocal` 选择恰当。扣分原因是存在注释掉的测试草稿文件、部分类缺少函数文档、`plugin.properties` 配置错误、`hotReload-v2` 分支的合并错误
- **Overall Assessment:** 7.5/10.0 - 热重载子系统的技术实现质量较高, 架构设计体现了对并发安全和资源回收的深入理解。Docker Compose 评审环境和跨平台脚本有效降低了复现成本。作者在 Rebuttal 中展现了诚实的自我反思和有效的改进行动。主要短板在于: 热重载仅影响分页大小未触及核心业务、demo 插件未涉及 IO 会话控制等核心难点、缺乏单元测试、`hotReload-v2` 分支存在编译错误
- **Confidence:** 4.0/5.0 - 我 (评审者) 熟悉 Java + Spring Boot + Maven 技术栈, 已成功完成 Maven 编译、Docker Compose 部署、配置热重载端到端验证、内置插件切换验证、外部 JAR 热加载验证。扣分原因是未能通过编写新插件验证 Level 2 的完整路径 (编译插件依赖主服务构建产物), 且对 `ClassLoader` 泄漏等深层问题的理解有限

面试参考问题 (困难难度)

1. (个人项目/架构设计) 简要介绍一下你博客项目的架构。
2. (个人项目/后端鉴权) 听说你的博客项目先后被入侵了两次, 你是怎么看待的? 现在项目的鉴权方式如何?
3. (个人技术栈/算法) 我听说你对算法有一定了解, 你能介绍一下你对红黑树, B+树吗、最小生成树的理解吗?
4. (个人技术栈/算法) 你是怎么看待算法, 数据结构的? 在日常开发过程中我们很少接触底层的算法实现, 大多数借助语言内置特性或者是第三方库帮我们实现这一部分。尽管如此你认为我们还要学习这类算法, 数据结构知识吗?
5. (个人技术栈/Java) 我在学习 Java Bean 的时候一直有一个疑问, 为什么我们通常要为 Bean 里面的属性进行私有化, 并且需要通过 `getter/setter` 函数访问这些私有化的数据, 难道我们不能直接设置为公共属性吗? (参考答案: 框架依赖/支持加入验证等逻辑/Java提供封装)
6. (个人技术栈/消息队列) 看到你的技术栈里面提到了 RabbitMQ, 你能介绍一下常见的消息队列框架吗? 他们底层是怎么实现的?
7. (第三轮考核/AI 使用) 你在第三轮考核中谈论了你的代码七成都是 AI 生成, 并且确保了对代码有审查。你能举一个 AI 生成的代码不符合你预期的例子吗? 你觉得我们应该怎么在不影响自己学习的基础上合理使用 AI 工具?
8. (**Hard**/第三轮考核/插件资源管理) 你在实现热重载的时候没有实现 I/O 控制, 抛开你交上来的项目不谈, 如果现在让你写一个支持热重载的插件系统, 你会怎么控制插件切换时的 I/O 会话? (参考答案: 定义一个接口来管理插件的 I/O 会话, 所有插件实现这个接口, 在插件切换的时候自动关闭 (处理长期会话, 比如 SQL); 统一资源接口, 规定插件必须使用框架提供的实现访问文件/网络/SQL I/O (处理短期会话, 比如文件/网络 I/O); 或者插件容器层做中心化管理, 容器统一负责关闭 I/O 会话。)

9. (从博客文章里挑的问题, 不会答说明博客文章用了 AI 生成) 在你博客里提到, Docker 部署 MySQL 并开启 binlog 时, 需要挂载配置文件并指定 `binlog_format` 为 `row`。请问, 为什么选择 ROW 格式, 而不是 STATEMENT?
10. (从博客文章里挑的问题, 不会答说明博客文章用了 AI 生成) 你博客里提到本地跑 JMeter 压测公网服务时, 有可能测到的是用户视角的端到端体验, 而本地网络波动可能会影响结果。请解释: 为什么本地→公网压测容易出现延迟异常或误判性能瓶颈? 如果要稳定测出系统自身容量, 应该怎么做? 请结合 JMeter 的部署和网络环境说明。(参考答案: **网络波动**: Wi-Fi/家宽高峰期丢包、延迟波动。**TLS/连接复用开销**: HTTPS 握手耗时影响请求响应。**本地端口/NAT 限制**: 高并发时端口耗尽, 导致连接排队或错误。**带宽瓶颈**: 本地上行带宽不足, 限制实际并发能力。解决方案: **使用同地域云主机**: 在与被测服务器同 VPC/同区域的 ECS 上部署 JMeter。

避免公网链路噪声, 测到的是系统真实处理能力。**分布式压测**: Master 控制多台 Slave 协同压测, 突破单机并发限制。**分阶段压测**: 冒烟测试 → 阶梯加压 → 长时稳定性验证。设置固定 Hold 时间, 避免短时波动导致误判。

李俊龙（隐藏高手）

251543236 - 计算机科学与技术（计算机学院）

Github: [xiaolongsysa \(小龙\)](#)

第二轮考核

[xiaolongsysa/Xiaolong-s-Backend-Assessment-Two](#)

同学你好, 你使用了 Spring Security 进行鉴权, 体现了对安全框架的实际掌握。采用 MyBatis-Plus 极大简化了 Mapper 层代码, 集成 Swagger/OpenAPI 为每个接口编写了参数说明, 接口文档意识非常好。连接了云端数据库、实现了周期性账单自动生成、预算管理和超支预警, 功能覆盖度达到了考核高级要求。同时尝试自己部署了服务器运行服务, 值得表扬。需要注意的是配置文件尽量不要包含敏感信息, 可以使用环境变量的方式避免敏感信息公开。总的来说, 完成度高, 基础扎实, 继续努力!

第三轮考核

Summary / 概述

项目实现了一个兼容（部分）OpenAI API 的后端服务, 技术栈为 Go + Gin + GORM + MySQL, 并附带一个 Vue3 + Vite + TypeScript 前端。

核心功能包括: Bearer Token 鉴权、Chat Completions（流式 SSE / 非流式）、生成结果生命周期管理（持久化 + GET/DELETE/CANCEL）、模型列表 API, 以及使用官方 OpenAI Python SDK 编写的集成测试。

可选加分项完成了三项: Files API（上传/列表/详情/删除, 含磁盘落库）、API 中转（多 provider 环境变量配置、确定性多 key 选择、自动回退）、Admin API（模型启用/禁用管理）。项目已部署至线上服务器并提供 Apifox 在线文档。

Rebuttal 后的补充评审

作者在收到初次反馈后进行了以下改进:

- 补充单元测试**（15 个 `*_test.go` 文件）: 覆盖 handler、middleware、repo、service、upstream、task 六个包, `go test ./...` 全部通过
- 添加 `.env.example`**: 降低本地配置门槛
- 完善 README**: 新增"快速开始 (TL;DR)"段落和单元测试章节
- 清理前端模板残留**: 移除 `counter.ts`、未使用的 `assets/*.png/.svg`、修复 `style.css`

Docker 实测结果（评审者编写 Docker Compose 在 WSL2 中部署）:

- 项目要求 Go ≥ 1.26.1（版本非常新），评审者通过 Docker Compose（MySQL 8.4 + Go 1.26 多阶段构建）成功部署
- 所有核心 API 验证通过: healthz、Chat Completions（非流式/流式 SSE 含参数回显）、Completion 生命周期（GET/DELETE）、Models 列表、Files API（上传/列表）、Admin API（模型启用/禁用）、鉴权拦截（无效 Token 返回 401）
- `go test ./...` 全部 6 个测试包通过
- 项目本身未提供 Docker Compose, 评审者为其编写了 `deploy/` 目录下的 Docker Compose 配置以实现一键部署

Strengths / 主要优点

- 架构分层清晰**: cmd → handler → service → repo → upstream 五层分离, 职责单一。Handler 层保持较薄, 业务逻辑集中在 service 层, 符合后端工程最佳实践。
- 错误处理一致性强**: 全项目统一使用 OpenAI 风格的错误体, 与官方规范对齐, 便于 SDK 客户端解析。
- API 中转实现精巧**: 多 provider 环境变量配置、FNV-1a 确定性哈希 key 选择、`UPSTREAM_FALLBACKS` 回退链机制、上游成功响应 ID 重写以保证生命周期一致性——这些细节体现了较强的工程抽象能力。

4. **Files API 边界情况处理扎实**：临时文件写入后 rename（原子性）、SHA-256 校验和计算、DB 写入失败时回滚磁盘文件，考虑了常见异常场景。
5. **流式输出实现正确**：真正的 SSE 逐块输出而非伪流式，mock 模式下每 chunk 间隔约 1 秒，支持中途 cancel 取消，且 cancel 信号正确传播到 context 链。mock 模板回显了 model、temperature、messages 数量和 chunk 序号，满足考核验收要求。
6. **文档可用性好**：提供线上 Apifox 文档（含令牌可直接测试）、已部署的演示站点、详细 README（架构、鉴权流程、环境变量说明）、OpenAPI 3.0.3 规范文件。
7. **前端功能完整且类型安全**：聊天（流式/非流式切换、`<think>` 标签增量解析、自动滚动）、模型管理（启用/禁用）、文件管理（上传/列表/删除）、移动端响应式适配。TypeScript 严格模式开启，零运行时依赖。
8. **部署至线上并作为一个有效的服务运行**：熟练运用考核内容对当前已有的项目进行扩展实现，是在众多学习者中较为缺失的特质，这一点值得肯定。
9. **Rebuttal 改进有效**：在收到反馈后补齐了 15 个测试文件，所有测试通过；添加了 `.env.example` 降低配置门槛；清理了前端模板残留代码。响应速度快、改进行动具体。

Weaknesses / 主要缺点

1. **缺乏 Docker 部署方案**：项目的 README 虽然补充了快速开始指南，但仍依赖本地 Go 环境和 MySQL。Go 1.26.1 版本非常新，评审者本地可能没有对应工具链。评审者为其编写了 Docker Compose 配置（`deploy/docker-compose.yml` + `deploy/Dockerfile.backend`）才实现了一键部署。建议项目作者将 Docker Compose 纳入交付物，降低评审与使用门槛。
2. **Admin API 完成度低**：考核文档要求的 Admin API 核心功能（添加/删除用户、分配/禁用 API Key、用户用量限制与统计）均未实现，仅完成了模型维度的启用/禁用管理，实际完成度约 30%。作者在 Rebuttal 中解释这是有意为之——希望先打好地基，用户管理作为后续升级功能。但在评审视角下，此部分确实未达考核标准。
3. **单元测试覆盖率有限**：虽然补充了 15 个测试文件是一大进步，但 `cmd`、`dto`、`model` 三个包无测试文件。测试覆盖集中在特定函数（如 upstream config 解析、fallback 判断），整体覆盖率仍不足 90%。且部分测试依赖真实数据库或上游服务，在 CI 环境中无法独立运行。
4. **函数文档不完整**：文档明确要求「每一个函数类都需要写明清晰的函数文档」。Repo 层（`completion_repo.go`、`ai_model_repo.go`、`file_repo.go`）的所有函数缺少函数级文档注释。此外 `file_service.go:176-180` 包含 `var _ = time.Second` 占位语句和无关注释，不应出现在交付代码中。
5. **前端完成度有限**：在本地部署测试中发现前端可用性较低。`App.vue`（1200+ 行）未拆分为独立组件，维护困难。虽然考核未要求前端完成度，但如果想作为一个有用的服务跑在服务器上，这一点需要重视。
6. **无系统与工程能力实现**：服务限流、最大并发控制、上下文复用与生成去重均为空白，虽然 README 诚实声明，但按考核标准该项不加分。
7. **mock 模板参数回显仍有改进空间**：当前 mock 回显了 model、temperature、messages 数量，但未回显实际的 messages 内容（如 user prompt 的具体文本）。考核文档要求“包含所有请求参数信息”。

Suggestions / 修改建议

1. **纳入 Docker Compose 部署**：评审者已为其编写了可用的 Docker Compose 配置（位于项目 `deploy/` 目录），建议将其纳入仓库并补充文档说明
2. **补全 Admin API**：至少实现用户的 CRUD 和 API Key 的分配/禁用功能，这是 Admin API 的核心价值所在
3. **提升单元测试独立性**：当前部分测试依赖 `MYSQL_DSN` 环境变量或外部服务。建议使用 `sqlite` 内存数据库（项目已依赖 `gorm.io/driver/sqlite`）或 mock 实现，使测试可以在纯 Go 环境中运行
4. **拆分 App.vue**：将 1200+ 行的单文件组件拆分为 ChatPanel、FileManager、ModelAdmin 等独立组件
5. **完善函数文档**：为 Repo 层所有函数补写文档注释；移除 `file_service.go` 中的占位语句
6. **改进 mock 模板**：在生成内容中回显完整的 messages 内容（或至少摘要），满足考核文档“包含所有请求参数信息”的要求

Questions for Authors / 给作者的问题

1. 为什么选用 Go + Gin 的技术栈完成此考核内容，技术栈的选择是否考虑了语言特性又或者仅仅是出于个人学习？**已由作者 Rebuttal 回答**：(1) 希望通过这次考核尝试使用 Go 完成一个完整的项目；(2) 项目有作为 API 中转站的功能，Go 语言的高并发特性会更适合这个工作。
2. 当前 Cancel 取消机制基于进程内 `map[string]*Task + context.CancelFunc`，多实例部署时 cancel 请求必须路由到持有任务的同一实例。如果要将此项目演进为可水平扩展的服务，你会如何改造 cancel 机制？**已由作者 Rebuttal 回答**：会根据建议尝试引入消息队列，任务队列化，cancel 变成对队列的控制消息。
3. Admin API 中用户管理（CRUD、API Key 分配、用量限制）未实现——这是时间不足还是有其他考虑？如果有更多时间，你计划如何设计用户与 API Key 的数据库模型关系？**已由作者 Rebuttal 回答**：(1) 希望先把项目的地基先做好，用户管理作为后续升级功能；(2) 数据模型：一个用户对应多个 API Key，每个 API Key 只属于一个用户，多个 API Key 对应同一个上游模型的 key。

4. 距离考核结束还有很长一段时间，为什么选择这个时间点提交？作者是否认为当前项目已经达到了最低可用标准？后续是否还有更多开发计划 **已由作者 Rebuttal 回答**：希望评审负责人先评估项目状态并提供建议；考核期间有许多比赛与事务要做，难以继续推动进度；会参考并吸取有用的建议进行完善修改。

Overall Assessment / 总体评分

- **Core features:** 4.5/5.0 - 很好地完成了项目最低要求实现，mock 模板参数回显基本满足要求（回显了 model、temperature、messages 数量），但未回显完整的 messages 内容
- **Extra features:** 1.5/2.0 - 完成了三项可选实现：Files API、API 中转（多 provider + 回退）、Admin API（模型管理部分）。Admin API 的核心用户管理功能缺失
- **Documentation completeness:** 1.5/2.0 - README 详尽度好、API 文档齐备、OpenAPI 规范文件完整。Rebuttal 后补充了快速开始指南和 `.env.example`。扣分原因是自述文件仍偏向考核文档风格，且缺少 Docker 部署指南
- **Code style:** 1.5/2.0 - 代码结构分层清晰、函数类命名规范、错误处理一致。但 Repo 层缺函数文档、部分后端实现存在未使用的占位语句、`App.vue` 1200+ 行未拆分
- **Overall Assessment:** 9.0/10.0 - 很好地完成了大多数考核要求，项目具有发展成实用 API 中转服务的潜力。Rebuttal 后补齐了单元测试和快速开始文档，响应积极。主要短板在于 Admin API 用户管理完全缺失、缺少 Docker 部署方案、Go 版本要求过新、前端组件未拆分
- **Confidence:** 4.0/5.0 - 评审者成功通过 Docker Compose 部署项目，验证了全部核心 API（healthz、Chat Completions 流式/非流式、Completion 生命周期、Models、Files、Admin），运行了完整测试套件且全部通过。对 Go + Gin 技术栈有一定了解但非深度专家

面试参考问题（困难难度）

1. （第三轮考核/实现点询问 - 用于判断是否完全 AI 生成）你的项目支持把 Chat Completions 请求转发到多个上游（MiniMax、火山引擎），还做了自动回退。能讲讲你的回退链是怎么工作的吗——从收到一个 completions 请求开始，系统怎么决定先调哪个 Provider、失败了怎么切换到下一个？什么样的失败会触发回退？
2. （继承第一题）如果上游只是响应慢但没有直接报错（比如卡了 30 秒），你的回退逻辑会怎么处理？当前代码有超时控制吗？
3. （第三轮考核/技术栈相关）你的项目里初始化数据库的函数用了 GORM 的 AutoMigrate，在启动时自动建表。AutoMigrate 很方便，但它有哪些做不了的事？如果上线后要加一个新字段，或者把某个字段从 varchar(64) 改到 varchar(128)，AutoMigrate 会自动处理吗？

参考答案：

1. AutoMigrate 能建表、加新列、但不会删除或修改已有列的类型(varchar(64)→varchar(128)不会生效),也不会删除未使用的列
2. 索引改名、字段重命名、表拆分等需要手动 SQL 或使用 Flyway/golang-migrate 这类专用迁移工具。AutoMigrate 的定位是原型和初期开发,生产环境通常不依赖它
3. 能指出项目里其实引入了 Flyway的 SQLite驱动(gorm.io/driver/sqlite)但没用到;能讨论 Go 生态中的迁移方案(golang-migrate+ embed 内嵌 SQL 文件)和CI/CD 中的迁移校验
4. （**Hard**/第三轮考核/语言底层原理）你提到起初用 Java 做第三轮考核，后来改用 Go。Go 的并发模型（goroutine + channel）和 Java 的线程模型（Thread / 线程池 / CompletableFuture）在设计哲学上有本质区别。你能分别说说他们的异同吗？
5. （**Easy**/第三轮考核）你提到你提交的项目已经部署在线上服务器。在你的整个服务体系中，这个项目主要负责什么部分？你觉得对你整体的服务，帮助大不大？
6. （第三轮考核/前端实现）留意到考核的前端部分，`createChatCompletionStream` 函数，SSE 流用了 fetch + ReadableStream 手写解析，而不是用浏览器的 EventSource。浏览器有原生的 EventSource API 可以直接处理 SSE，不需要手动读 ReadableStream 和解析 data: 行。你为什么没用？如果用了，会遇到什么问题？

参考答案：

1. EventSource 只支持 GET 请求，而 Chat Completions 需要 POST（带 JSON body），且 EventSource 无法设置自定义请求头（如 Authorization），所以必须用 fetch 手动实现
2. 能解释 TextDecoder 的 stream: true 参数的用途——告诉解码器"后面还有数据"，避免多字节 UTF-8 字符（如中文、emoji）被截断在半中间时产生乱码（）。buffer 的作用是处理 TCP 分片：一个 data: {...}\n\n 可能被切成两次 reader.read() 返回
3. 能讨论 EventSource 的自动重连机制与手动 fetch 的差异：EventSource 断线自动重连，fetch 需要手动实现重试逻辑。能进一步谈到：你在这段代码里没有做重连——如果流式请求中途断开，前端会直接抛异常，这是当前的局限

朱涛（基础扎实）

第二轮考核

[DUSKDESU/GDUF-Web-Account](#): 这是GDUF爪哇部后端组第二轮考核的项目

同学你好，账单相关接口的 URL 设计具备了 RESTful 意识，也完成了基本的功能，使用了 MyBatis 注解写 SQL，但是包结构有一些问题，建议学习一下 Java 项目规范。你的包名以大写字母开头，违反了 Java 包命名全小写的约定，同时缺少 Service 层，建议学习一下 MVC 分层原则。能完成基本的登录注册和账单增删改查，说明对 Spring Boot 和 MyBatis 有了初步掌握，希望继续学习 Java 包规范和三层架构，基础打牢之后会进步很快！

第三轮考核

Summary / 概述

本项目实现了一个功能全面的 OpenAI API 风格后端服务，技术栈为 Java 17 + Spring Boot 3.2.5 + Spring Security + JWT + Spring Data JPA + MySQL + Docker Compose。项目规模在已评审的 Level 3 提交中名列前茅：7 个 Controller、5 个 Service 接口 + 实现类、5 个 JPA Entity、5 个 Repository、49 个 Java 源文件。核心功能覆盖：用户注册/登录（JWT 鉴权 + BCrypt 加密）、API Key 管理（创建/启用禁用/配额/速率限制）、Chat Completions（异步 `DeferredResult` + `CompletableFuture` + RestTemplate 真实 API 转发）、Completion 生命周期管理（GET/DELETE/CANCEL）、Models API（代理到阿里云 DashScope）、Files API（上传/列表/详情/删除）、Admin API（用户 CRUD/状态切换/模型管理）。集成 SpringDoc OpenAPI 3 提供 Swagger UI。提供 Docker Compose 一键部署（MySQL 8.0 + 应用容器）。经 Maven 编译验证：49 个源文件编译通过（`BUILD SUCCESS`），项目架构完整可用。

Strengths / 主要优点

- 项目规模与功能覆盖全面**：49 个源文件，涵盖认证、鉴权、API Key 管理、OpenAI 代理转发、文件管理、管理后台六大模块。实现了一整套完整的 API 服务体系，远超 MVP 最低要求。
- 安全设计成熟**：
 - 双通道鉴权：JWT（用户登录/后台管理）+ API Key（OpenAI 兼容端点），分工明确
 - 密码使用 BCrypt 加密（`PasswordEncoder` Bean）
 - JWT 使用 HS512 签名，24h 访问令牌 + 7d 刷新令牌
 - Spring Security + `OncePerRequestFilter` 过滤器链，无状态会话
 - RBAC 权限控制：`ROLE_ADMIN` vs `ROLE_USER`
 - API Key 使用 `SecureRandom` 生成 50 字符 `sk-` 前缀密钥，支持屏蔽显示（`maskApiKey()`）
- 真实 API 转发实现正确**：`CompletionServiceImpl.callRealOpenAI()` 通过 `RestTemplate` 调用 OpenAI 兼容端点，包含多种异常处理（HTTP 4xx/5xx、HTML 错误页面检测、空响应检测）、处理时间统计、token 用量记录。上游 URL 和 Key 通过环境变量配置，支持任意 OpenAI 兼容提供商。
- 异步处理架构**：`OpenAIProxyController` 使用 `DeferredResult` + `CompletableFuture.supplyAsync()` 实现异步非阻塞请求处理，避免了 Tomcat 线程阻塞。
- 数据库设计规范**：5 个 JPA Entity（User/APIKey/Completion/Model/File）均使用标准注解，包含完整字段（时间戳、状态枚举、外键关联）。`Completion` 记录了请求 JSON、响应内容、token 用量、处理时间等完整审计信息。
- 配额与限流机制**：API Key 级别支持每日请求上限（`dailyLimit`）、速率限制（`rateLimit`），每次调用后递增使用计数（`incrementApiKeyUsage()`）。Completion 级别有配额检查（`checkQuotaLimits()`，硬编码 100 次/天）。
- Docker Compose 部署完备**：提供多阶段 Dockerfile（Maven 构建 + JRE Alpine 运行镜像，创建非 root `appuser`）和 `docker-compose.yml`（MySQL 8.0 + 应用，健康检查，网络隔离，卷持久化）。环境变量全面外部化。
- Swagger/OpenAPI 文档**：集成 SpringDoc OpenAPI 3，`SecurityConfig` 中配置了 Bearer JWT 安全方案，所有 Controller 均标注了 `@Tag` 和 `@Operation` 注解。
- DTO 分层规范**：请求/响应 DTO 分离（`dto/request/` + `dto/response/`），`ApiResponse<T>` 泛型统一响应封装（`success/message/data/error/timestamp`）。`ChatCompletionRequest` 字段完整（`model/messages/temperature/stream/max_tokens/top_p/stop/penalty`），与 OpenAI 规范高度一致。
- 工程细节到位**：Lombok `@Data` 减少样板代码、Jakarta Validation 参数校验、`spring.jpa.open-in-view=false` 避免性能陷阱、Actuator 健康检查端点、文件上传 100MB 限制、UUID 文件名防冲突。

Weaknesses / 主要缺点

1 MVP 关键缺失

- 无 SSE 流式输出**：考核文档要求 Chat Completions 支持流式（`stream=true`）。`ChatCompletionRequest` DTO 虽然有 `stream` 字段，但 `CompletionServiceImpl.createChatCompletion()` 和 `callRealOpenAI()` 完全不处理流式——仅使用 `RestTemplate.exchange()` 获取一次性响应（非流式）。`simulateOpenAIResponse()` 也是非流式 mock。这是 Level 3 MVP 的硬性缺失。

2. **无 Python SDK 测试脚本**：考核文档要求「使用官方 OpenAI Python SDK 编写测试脚本」。项目中无任何 `.py` 文件。README 中有 `curl` 命令示例但非 SDK 测试。

2 安全与代码缺陷

3. **FileController.getCurrentUserId() 硬编码返回 1L**：所有文件操作（上传/列表/详情/删除）均绑定到 `userId=1`，完全绕过身份验证。无论哪个用户登录，文件操作的归属均为同一用户。这是一个明显的未完成占位实现（TODO）。
4. **docker-compose.yml 暴露真实 API 密钥**：`OPENAI_API_KEY` 的默认值是一串看似有效的 `sk-a676b8eb...` 密钥，`DB_PASSWORD` 和 `JWT_SECRET` 也以明文硬编码在 `compose` 文件中。即使使用了 `${VAR:-default}` 语法，这些默认值已被提交到 Git 仓库，存在密钥泄露风险。
5. **cors.disable() 与 CorsConfig 存在冲突**：`SecurityConfig` 第 52 行 `.cors(cors -> cors.disable())` 禁用了 Spring Security 的 CORS 处理，但项目又定义了 `CorsConfig` Bean（允许所有来源的宽松配置）。这两种配置互相矛盾，可能导致 CORS 行为不可预测。
6. **Model.java 列注解错误**：`@Column(name = "is_Active")` 使用了错误的大写 `A`（第 34 行）。SQL DDL 和 MySQL 实际列名应为 `is_active`（全小写）。`@Column(name = "displayname")` 缺少下划线（第 26 行），SQL 中为 `display_name`。这些不一致会导致 JPA 映射异常。

3 功能不完整

7. **Refresh Token 端点为存根**：`AuthController.refreshToken()` 接收刷新令牌但返回 `ApiResponse.success("令牌刷新成功", null)`——`data` 字段为 `null`，不返回新令牌。功能完全未实现。
8. **Cancel 机制为伪实现**：`CompletionServiceImpl.cancelCompletion()` 仅将数据库状态改为 `cancelled`，但 `CompletableFuture` 没有实际取消机制——正在进行的 API 调用不会被中断。真正的 `cancel` 需要 `future.cancel(true)` 或 `RestTemplate` 超时配置。
9. **无 mock 回退**：`simulateOpenAIResponse()` 方法已定义但未集成到主流程中。当 `OPENAI_API_KEY` 未配置时，`callRealOpenAI()` 会产生 HTTP 错误而非回退到 mock。评审者无法在不配置真实 API Key 的情况下验证聊天功能。
10. **仅有一个单元测试**：`contextLoads()` 验证 Spring Context 启动，无任何业务逻辑测试、无鉴权测试、无 API 端点集成测试。

4 代码细节

11. **Quota 硬编码 100 次/天**：`checkQuotaLimits()` 中 `todayCount >= 100` 硬编码在代码中，未通过配置文件或数据库管理。
12. **Package 命名 test.openai 不自然**：项目顶层包名为 `test.openai`，在 Spring Boot `@ComponentScan` 下可以正常工作但不符合常见的 `com.example.projectname` 约定。
13. **FileUploadRequest 和 ModelResponse DTO 已定义但未使用**：死代码残留。
14. **FileController 响应类型不一致**：`listFiles()` 返回 `ResponseEntity<ApiResponse<FileResponse>>` 但列表应返回包含 `data` 数组的包装类型。
15. **ChatCompletionRequest.Message 类缺失**：需要确认是否存在独立 DTO（已在 `ChatCompletionRequest.java` 中找到内部类实现）。

Suggestions / 修改建议

1. **实现 SSE 流式输出（最高优先级）**：使用 `Spring SseEmitter` 或 `WebClient` 实现 `stream=true` 的逐 token SSE 输出。这是 Level 3 最关键的缺失功能
2. **编写 Python SDK 测试脚本**：创建 `tests/sdk_test.py`，使用 `openai` Python 库覆盖 `models.list`、`chat.completions.create`（非流式）、`chat.completions.create`（流式）
3. **修复 FileController 用户绑定**：通过 `SecurityContextHolder.getContext().getAuthentication()` 获取当前登录用户 ID，而非硬编码
4. **清除 docker-compose.yml 中的真实密钥**：将所有默认值替换为明显的占位符（如 `sk-placeholder`），真实密钥仅通过 `.env` 注入（将 `.env` 加入 `.gitignore`）
5. **集成 mock 回退**：当 `OPENAI_API_KEY` 为空或 API 调用失败时，回退到 `simulateOpenAIResponse()`
6. **补充单元测试**：优先覆盖 `AuthController`（注册/登录/Token 验证）、`ApiKeyService`（生成/校验/配额）、`CompletionService`（状态机转换）
7. **实现真正的 Cancel**：将 `CompletableFuture` 保存到 `ConcurrentHashMap`，`cancel` 时调用 `future.cancel(true)`
8. **修复 Model.java 列注解**：`is_Active` → `is_active`、`displayname` → `display_name`
9. **实现 Refresh Token**：验证刷新令牌后生成新的 access token 并返回

Questions for Authors / 给作者的问题

1. `FileController.getCurrentUserId()` 硬编码返回 `1L`，这意味着所有文件操作都绑定到同一个用户。这是一个已知的未完成占位还是代码遗漏？你计划如何获取当前登录用户的 ID？
2. `ChatCompletionRequest` DTO 中有 `stream` 字段，但 `CompletionServiceImpl` 完全不处理流式模式。你是否有计划实现 SSE 流式输出？如果有，你打算使用 Spring 的哪个机制（`SseEmitter`、`Flux`、还是其他）？
3. `SecurityConfig` 中 `.cors(cors -> cors.disable())` 和 `CorsConfig` Bean 同时存在——一个是禁用 CORS，一个是开启宽松 CORS。实际运行中哪个配置生效？你是否有遇到 CORS 相关的问题？
4. 在 `Info.md` 中你提到通过 AI 辅助（千问）完成了项目中重复功能的体力活代码。你如何确保 AI 生成的代码（尤其是安全相关代码如 JWT、鉴权）的正确性？在人工审查中你是否发现了 AI 代码的任何问题？

Overall Assessment / 总体评分

- **Core features:** 3.0/5.0 - Chat Completions 实现了真实 API 转发（含异步处理、错误处理、用量记录）、Models API（代理到 DashScope）、Bearer Token/JWT 鉴权正确、Completion 生命周期完整（GET/DELETE/CANCEL）、数据库持久化。扣分原因：无 SSE 流式输出（仅 `RestTemplate.exchange` 一次性响应）、无 Python SDK 测试脚本、FileController 用户 ID 硬编码导致文件归属错误、Cancel 为伪实现（仅改数据库状态不实际中断 API 调用）
- **Extra features:** 1.5/2.0 - 超额完成了多项可选功能：Files API（上传/列表/详情/删除）、Admin API（用户管理/模型管理）、API 中转（真实 RestTemplate 转发到 OpenAI 兼容端点）、系统与工程能力（Docker Compose、JWT + API Key 双通道鉴权、配额限流、RBAC、Swagger 文档）。扣分原因：API 中转无流式支持、无多 provider 回退链、API Key 配额硬编码
- **Documentation completeness:** 1.5/2.0 - README 详尽（架构图、鉴权流程、SDK 测试指南、API 列表、Docker 部署）、AIGC 披露适度。扣分原因：缺少 Python SDK 测试脚本（使用 curl 替代）、Swagger 文档依赖应用启动后访问（无导出静态文件）
- **Code style:** 1.5/2.0 - 分层架构规范（Controller→Service接口→ServiceImpl→Repository）、DTO 分离请求/响应、Lombok 合理使用、Jakarta Validation 参数校验、`ApiResponse<T>` 统一响应。扣分原因：`FileController` 硬编码用户 ID、`Model.java` 列注解不一致、`FileUploadRequest / ModelResponse` 死代码、包名 `test.openai` 不规范、`cors.disable()` 与 `CorsConfig` 冲突、仅一个单元测试
- **Overall Assessment:** 7.5/10.0 - 项目的整体工程完成度在已评审提交中较高：49 个源文件、完整的用户+鉴权+API Key+管理后台+文件系统、Docker 部署、Swagger 文档。作者展现了将 Spring Boot 生态组件（Security、JPA、RestTemplate、DeferredResult）组合为完整后端服务的能力。主要短板在于两项 MVP 硬性要求（SSE 流式 + Python SDK 测试）的缺失，以及 `FileController`、`Model.java` 等细节未完善。如果能补上流式输出和 SDK 测试，这将是一个非常优秀的项目提交
- **Confidence:** 3.5/5.0 - 评审者熟悉 Java + Spring Boot 技术栈，已成功编译全部 49 个源文件。由于自身网络原因未能通过 Docker Compose 完整部署和测试全部端点（MySQL 镜像下载过慢），API 功能验证基于代码审查而非实际运行。对 Spring Security + JWT 组合有实操经验，对 RestTemplate 异步处理模式有理解

面试参考问题(中等难度)

(wip)

王秉桦

251731619 - 计算机科学与技术(大数据与人工智能学院)

第三轮考核（不及格）

[afatpitb/openai-api-service: Mock OpenAI API](#)

Summary / 概述

本项目实现了一个简化版 OpenAI API 风格后端服务，技术栈为 Java + Spring Boot + Spring Data JPA + H2 内存数据库。核心功能覆盖 Chat Completions（非流式/流式）、Models API（硬编码两个模型）、Bearer Token 鉴权、Guava RateLimiter 限流（5 req/s），以及 Completion 生命周期管理。

初次反馈后的更新 (3630ec8..40177fb)

作者在收到初次反馈后进行了代码修改，提交记录为 `1deeb56` + `40177fb` (merge)。主要变更包括 13 个文件的增删改（+333 / -328 行）：

正向改进：

- `pom.xml` 修复：Spring Boot `4.0.6` → `3.3.2`（正确版本），JDK `25` → `17`，非标准 starter 名称修正（`spring-boot-starter-webmvc` → `spring-boot-starter-web`），Guava 重复版本合并
- `AuthInterceptor.java` 代码逻辑修复：从“任意非空 Token 通过”改为白名单校验（`api.tokens` 配置驱动），代码已重新格式化

- `application.properties` 修改：移除 `OPENAI_API_KEY` 环境变量强制依赖，添加 `api.tokens=test-token`
- 删除死代码：`ChatRecord.java`、`InvalidRequestException.java`、`DatabaseException.java`、`JacksonConfig.java`、`WebConfig.java`
- `OpenaiController.java` 重构：添加模型白名单校验（`ALLOWED_MODELS`）、非流式模式 mock 回退、代码格式化
- `ChatCompletionService.java` 增强：流式模式支持真实 OpenAI API 调用 + mock 回退、非流式模式异常回退到 mock、修复只发送首条消息的 bug（现在遍历全部 messages）
- 新增 `src/test/sdk_test.py`：OpenAI Python SDK 测试脚本（models / chat 非流式 / chat 流式）

引入的新缺陷（严重）：

- `WebConfig.java` 被删除但无替代实现 → `AuthInterceptor` 从未被注册到 Spring MVC 拦截器链，**鉴权完全失效**，所有请求无论携带什么 Token（甚至无 Token）均通过
- `application.properties` 中 `${openai.api.key:}` 是循环占位符引用 → 应用启动时崩溃（`IllegalArgumentException: Circular placeholder reference`）
- `GlobalExceptionHandler.java` 被删除但无替代 → 验证异常不再返回 OpenAI 风格错误体，且 5xx 异常处理完全丢失

Strengths / 主要优点

1. **分层架构清晰**：Controller → Service → Repository → Entity 四层结构符合 Spring Boot 惯例，职责边界明确。
2. **参数校验完整**：ChatRequestDTO 使用 Jakarta Bean Validation，对非法输入返回结构化 OpenAI 风格错误体。
3. **SSE 流式实现正确**：使用 Spring `SseEmitter` 逐词输出，支持中途 cancel；流式返回数据格式正确。
4. **限流有基本实现**：Guava `RateLimiter.create(5.0)` 控制 Chat Completions 端点的请求频率，超限返回 429 + OpenAI 风格错误体。
5. **CORS 支持**：`AuthInterceptor` 中自动写入 CORS 响应头（动态 Origin），对 OPTIONS 预检请求友好处理。
6. **Mock 流式输出包含请求参数回显**：model、temperature、messages 数量、最后一条 user 消息内容均出现在生成文本中，便于验证接口行为。
7. **积极采纳反馈意见**：作者在短时间内对 pom.xml、鉴权逻辑、死代码清理、SDK 测试脚本、ChatCompletionService 等多个方面进行了大量修改，表明对反馈意见的重视和较强的行动力。

Weaknesses / 主要缺点

1 安全/功能缺陷（严重）

1. **鉴权完全失效（新引入）**：`webConfig.java` 被删除，但 `AuthInterceptor` 没有被重新注册到 Spring MVC 拦截器链。虽然 `AuthInterceptor.java` 本身的鉴权逻辑已正确实现（白名单校验），但因未注册，拦截器永远不会被调用。**任何请求——无论携带无效 Token 还是完全不携带 Token——均能通过并成功调用 API**。经实际测试验证：`Authorization: Bearer any-random-token` 和完全不携带 Authorization 头均可正常返回数据。与初次提交的“任意非空 Token 通过”相比，**问题反而更严重了**（现在空 Token 也通过）。
2. **应用启动崩溃（新引入）**：`application.properties` 中 `openai.api.key=${openai.api.key:}` 导致循环占位符引用，Spring 启动时抛出 `IllegalArgumentException: Circular placeholder reference 'openai.api.key:' in property definitions`。正确写法应为 `${OPENAI_API_KEY:}`（引用环境变量而非自身）或直接留空。经实际测试：修改此行为 `openai.api.key=` 后方可正常启动。
3. **5xx 错误处理完全丢失（新引入）**：`GlobalExceptionHandler.java` 被删除但无替代方案。虽然这消除了旧版的堆栈跟踪泄露问题，但也导致所有未捕获异常走 Spring Boot 默认错误处理，不再返回 OpenAI 风格错误体。

2 代码质量问题

4. **Raw type 使用**：`ChatCompletionService.java:148` 仍使用未经泛型参数化的 `Optional`（应为 `Optional<ChatCompletion>`），编译时产生 unchecked warning。**未修复**。
5. **5xx 错误泄露完整堆栈跟踪**：**部分修复——**`GlobalExceptionHandler` 被删除消除了直接泄露路径，但也丢失了所有结构化错误处理。

3 功能缺失

6. **模型列表硬编码**：`ModelsController` 中仍直接 hardcode `gpt-4o-mini` 和 `mock-echo-001` 两个模型，无数据库持久化。**未修复**。
7. **Model 参数不校验**：**已修复——**新增 `ALLOWED_MODELS` Set 校验，无效模型名返回 400 + OpenAI 风格错误体。**未修复**。
8. **限流范围不完整**：`RateLimiter` 仅在 `POST /v1/chat/completions` 生效，Models API 和 Completion 管理接口无限流，可通过这些端点绕过限制。
9. **H2 内存数据库**：数据在应用重启后全部丢失，不符合「使用数据库持久化」的实际意义。

10. 仅一个单元测试: `OpenaiApiServiceApplicationTests.contextLoads()` 仅验证 Spring Context 能启动, 不测试任何业务逻辑。
11. 无任何可选加分项完成: Files API、系统与工程能力 (限流有但鉴权失效导致无意义)、Admin API、API 中转均未实现。

Suggestions / 修改建议

1. 修复鉴权注册 (最高优先级): 恢复 `WebConfig.java` (或等效的 `WebMvcConfigurer` 实现), 将 `AuthInterceptor` 注册到 `/v1/**` 路径。这是三个提交新的致命缺陷中最关键的一个
2. 修复循环占位符: 将 `application.properties` 中的 `openai.api.key=${openai.api.key:}` 改为 `openai.api.key=${OPENAI_API_KEY:}` 或 `openai.api.key=`
3. 恢复全局异常处理: 重新添加 `@RestControllerAdvice`, 保持 OpenAI 风格错误体但只返回通用错误消息 (不泄露堆栈)
4. 统一 mock 输出格式: 非流式 mock 和流式 mock 应返回一致的参数回显内容
5. 补充单元测试: 至少为 `ChatCompletionService` 和 `AuthInterceptor` 编写单元测试
6. 完善 README: 更新环境变量说明 (`api.tokens` 替代了硬编码), 移除已不存在的 `OPENAI_API_KEY` 强制要求
7. 修复 Raw type: 将 `Optional` 改为 `Optional<ChatCompletion>`

Questions for Authors / 给作者的问题

1. `AuthInterceptor` 中注释 `// MVP: token 非空即视为有效`——你如何理解考核文档中「未携带或非法 Token 的请求应返回 401-Unauthorized」这句话? “非法”Token 的定义是什么? 作者已通过代码修复回答: 将鉴权逻辑改为白名单校验, 但 `WebConfig.java` 被误删导致拦截器未注册。请确认: 删除 `WebConfig.java` 是有意为之 (打算换一种注册方式但未完成) 还是清理代码时的误删?
2. 接上述问题, 整体来看你的项目完成度较低, 而距离截止日期还有一段时间。作为第一位提交的作者, 你选择在这个时间点上提交的目的是什么? 为知道当前项目的问题所在还是能力所限? 作者未提交 Rebuttal, 此问题未答复。
3. `pom.xml` 中使用子 `spring-boot-starter-parent 4.0.6` 和 `maven-compiler-plugin source/target 25`, 这是如何确定的? 项目实际在哪些 Java 版本上测试通过? 作者已通过代码修复回答: 改为 `3.3.2` 和 `17`, 表明之前的版本号是错误的。
4. `ChatRecord` 和 `ChatCompletion` 两个 Entity 的区别是什么? `ChatRecord` 是否有未完成的设计意图? 作者已通过代码修复回答: 删除了 `ChatRecord.java`, 表明它确实是冗余的死代码。
5. 为什么非流式模式选择调用真实 OpenAI API, 而流式模式使用 mock? 这是一致的设计决策还是时间原因导致的分叉? 作者已通过代码修复回答: 两种模式现在都支持真实 API 调用 + mock 回退。
6. (新增) `application.properties` 中 `openai.api.key=${openai.api.key:}` 导致了启动崩溃, `${openai.api.key:}` 和 `${OPENAI_API_KEY:}` 之间有什么区别? 你在提交前是否测试过 `mvn spring-boot:run` 或 `java -jar` 启动?
7. (新增) 你在更新中删除了 `WebConfig.java` 和 `GlobalExceptionHandler.java`, 但没有添加替代实现。你能描述一下这两处删除的设计意图吗? 是打算完全重构这些模块但未完成, 还是在清理代码时没有意识到这些类的功能角色?

Overall Assessment / 总体评分

- **Core features:** 2.5/5.0 - MVP 各项在代码层面存在但质量严重不足: 鉴权逻辑已修复但未注册 (完全失效), 应用无法直接启动 (循环占位符), 非流式模式 mock 回退已实现但引入新问题。积极的一面是 SDK 测试脚本已补上、`pom.xml` 已修复、死代码已清理
- **Extra features:** 0.5/2.0 - 仅实现限流 (但鉴权失效导致限流意义有限), 未实现其他可选功能。与初次提交相比无变化
- **Documentation completeness:** 1.0/2.0 - README 覆盖功能列表但未随代码更新同步修改 (如 `api.tokens` 替代环境变量、不再需要 `OPENAI_API_KEY` 等信息未更新)。与初次提交相比无变化
- **Code style:** 1.0/2.0 - 死代码已清理、`pom.xml` 已规范、代码格式化已改善。但 raw type 未修复、全局异常处理器被误删、新引入的两个致命缺陷 (鉴权未注册 + 循环占位符) 表明代码质量审查不足
- **Overall Assessment:** 5.0/10.0 - 作者的修改方向总体正确: 鉴权逻辑、死代码清理、`pom.xml` 修复、SDK 测试脚本都是应该做的改进。但由于两个新引入的致命缺陷 (鉴权未注册 + 启动崩溃), 项目质量出现了先退步再进步的局面——修复了一部分问题, 但新增了更严重的问题。这表明作者在提交前未进行充分的集成测试 (如果启动一次 `java -jar` 就会立即发现循环占位符导致的崩溃)
- **Confidence:** 4.0/5.0 - 评审者熟悉 Java + Spring Boot 技术栈, 已成功编译、发现并定位两个新缺陷 (循环占位符 + 拦截器未注册), 通过修正循环占位符后验证了所有 API 端点。扣分项主要是因为未能验证鉴权修复的实际效果 (因鉴权根本未生效)

面试参考问题 (简单难度)

(wip)

陈雅莉

第二轮考核

<https://gitee.com/fir-velo/personal-accounting-web>

你在技术选型上展现出了不错的眼界，选用 FastAPI + SQLAlchemy 的组合并初步搭建了 MVC 分层结构

(api/models/crud/schemas)，说明你已经对后端工程化有了一定的认知，这是很好的起点。不过项目目前还存在一些需要重点改进的地方：`[auth.py]`(<http://auth.py>) 中 `/login` 路由被二次定义且错误嵌套在 `if not user` 条件块内，导致 Token 生成逻辑永远不会被执行，登录功能目前处于不可用状态，建议优先排查修复；`[bills.py]`(<http://bills.py>) 和 `[users.py]`(<http://users.py>) 中所有数据库操作均硬编码了 `user_id=1`，没有接入认证中间件，这会造成任何人不登录就能操作他人账单的越权问题，安全上需要特别注意；主目录下的 `[database.py]`(<http://database.py>) 和 `[config.py]`(<http://config.py>) 均为空文件，真正的代码却放在了 [.vscode](#) (IDE 配置目录) 下，导致项目实际无法运行，希望你整理一下工程目录结构；此外，字段命名 `typee` 是规避关键字的非标准写法，可以改用 `type_` 或枚举类型来更优雅地处理；响应格式也不统一，有的返回 `{"success": True}`，有的直接裸返回 ORM 对象，建议统一封装；功能完成度上，账单详情查询、总余额统计、管理员模块、修改个人信息及密码等均尚未实现，`[main.py]`(<http://main.py>) 中 `bills.router` 也被重复注册了两次导致路由冲突，这些都值得一一补齐。总的来说，你已经迈出了从单机程序到 Web 服务的第一步，框架思路是对的，接下来如果有时间，建议先把认证鉴权流程跑通、将 `get_current_user` 依赖注入到需要保护的路由中，再逐步补全功能，并在提交前自己克隆一遍仓库验证能否正常运行——养成这个习惯会让你的工程能力提升得更快。

面试参考问题（简单难度）

(wip)

郑都裕

251543610 - 计算机科学与技术(计算机学院)

第一轮考核

<https://github.com/zheng0311-ops/java-accounting-app.git>

学弟学妹你好，看了你的代码，你用纯 Java SE（没用任何框架和第三方库）写了一个命令行记账程序，实现了注册登录、账单的增删改查、按类型统计和总余额查询这些功能，把 `user` 和 `Bill` 抽成了两个类，构造成面向对象的形式，作为 Level 1 的难度来说思路是对的。

不过有几个比较关键的问题。最核心的是**完全没有数据持久化**——所有用户、账单都在 `static ArrayList` 内存里存着，程序一关数据全没。考核文档里写了要用 JSON 或文件保存数据，这是 Level 1 的基本要求之一。其次是密码明文存储（`User.java` 里 `password` 就是最普通的 `String`），至少应该做一层 SHA-256 加盐，不然万一数据文件被看到了所有密码就暴露了。代码层面有个 bug 在 `statByType()` 方法里——`totals.add(index, totals.get(index) + b.getAmount())` 这行用的是 `add` 而不是 `set`，`add(index, element)` 是在指定位置插入新元素并右移后面的元素，但你需要的是替换当前位置的值，用 `set(index, ...)` 才对，这会导致统计结果出错。还有一个问题，`addBill()` 里 `int id = currentUser.getBills().size() + 1` 来生成 ID——删掉一条记录后 `size` 变小了，新账单可能会和已有的拿同个 ID。另外 `Scanner.nextInt()` 和 `nextDouble()` 没有任何 try-catch 保护，输入字母就会崩。`updateBill()` 用了 `remove + add` 的方式更新，不如直接调 `setter` 改字段更合理。README 里放了界面截图这一点挺好的，但整个项目没有构建工具（没有 `pom.xml`），编译依赖手动 `javac`，也没有 `.gitignore`。

整体来说，项目具备了最基本的 CRUD 骨架，但核心的数据持久化完全缺失，代码里也还有几个比较明显的 bug 和安全性问题。如果后续有时间改进的话，建议最优先把文件读写加上（用 `BufferedWriter/Reader` 写 JSON 不算难），然后给密码做哈希，修掉 `stats` 的 `add/set` 问题，加个 try-catch 处理输入异常。如果能进入面试环节的话祝你一切顺利，学业进步！

第三轮考核（不及格）

<https://github.com/zheng0311-ops/openai-api-recruit03>

Summary / 概述

本项目实现了一个 OpenAI API Mock 服务，技术栈为 Java 17 + Spring Boot 3.2.6 + Spring Data JPA + H2 内存数据库。核心功能覆盖 Chat Completions（非流式 + 流式 SSE `SseEmitter`）、Models API（硬编码两个模型）、Bearer Token 鉴权（HandlerInterceptor 拦截器）、Completion 生命周期管理（GET/DELETE/CANCEL），以及一个 Python SDK 测试脚本。经编译和启动测试，应用可正常运行，核心 CRUD 端点均可工作。

Strengths / 主要优点

- 分层架构清晰**：Controller → Repository → Entity 三层分离，Interceptor 鉴权独立于业务逻辑，`webConfig` 正确注入 Spring 管理的拦截器（`@Autowired` 而非 `new`）。
- SSE 流式实现正确**：使用 `Spring SseEmitter + new Thread()` 异步执行，逐字符推送（50ms 间隔），返回格式包含 `delta.role` 初始化和 `finish_reason: "stop"` 结束标记。非流式响应格式与 OpenAI 规范一致（含 `usage` 字段）。

3. **鉴权拦截器设计正确**：AuthInterceptor 从配置文件读取合法 Token (@value)，使用 Objects.equals() 防 NPE，返回 OpenAI 风格 401 错误。鉴权路径通过 WebConfig 精细控制。
4. **JPA 持久化工作正常**：ChatCompletionRecord Entity 包含完整字段 (id/model/requestMessages/temperature/stream/responseContent/isCancelled/createdAt)，JpaRepository 自动提供 CRUD 操作。
5. **Python SDK 测试脚本**：使用官方 openai Python 库编写了非流式测试 (client.chat.completions.create)，验证响应 ID 和内容。
6. **README 文档详尽**：覆盖功能列表、环境要求、运行步骤、API 示例、项目心得。技术选型说明清楚。
7. **积极的代码演化**：ChatController.java 顶部保留了三个版本的代码演进痕迹 (第一版基础 mock → 第二版固定响应 → 第三版完整实现)，虽然有清理空间，但体现了迭代学习的痕迹。

Weaknesses / 主要缺点

1 安全与功能缺陷

1. **鉴权覆盖不完整**：WebConfig.java:36 仅对 /v1/chat/completions 路径注册了拦截器。GET /v1/models (ModelController)、GET /v1/chat/completions/{id} (历史查询)、DELETE /v1/chat/completions/{id} (历史删除)、POST /v1/chat/completions/{id}/cancel 均无鉴权保护。**经实测验证**：不带有效 Token 即可访问 /v1/models 成功返回模型列表。不符合考核文档「所有 /v1 端点必须鉴权」的要求。
2. **Cancel 端点路径错误**：ChatController.java:326 — @PostMapping("/v1/chat/completions/{id}/cancel") 在类级别 @RequestMapping("/v1/chat") 下，实际完整路径为 /v1/chat/v1/chat/completions/{id}/cancel。路径前缀被重复拼接。正确写法应为 @PostMapping("/completions/{id}/cancel")。经分析，此端点在实际使用时将返回 404。
3. **Cancel 为伪实现**：cancelCompletion() 仅将 isCancelled 字段设为 true 后写库，不中断正在执行的 SSE Thread。流式线程将继续完整输出直到结束。不符合考核文档要求的“取消正在处理的请求”。
4. **Thread.sleep(1000) 阻塞主线程**：ChatController.chatCompletions() 第 196 行在返回响应前 Thread.sleep(1000)。对于非流式请求，这直接阻塞 Tomcat worker 线程 1 秒；对于流式请求，该延迟在 SSE 线程启动前执行，同样不必要。高并发下会耗尽线程池。

2 代码质量问题

5. **~150 行注释掉的旧代码**：ChatController.java 第 1-149 行包含 "First" 和 "Second" 两个完整版本的注释代码。已交付的项目应移除或归档到独立分支。
6. **AuthInterceptor.java 同样有注释掉的旧版**：第 1-30 行为旧版注释代码。
7. **死代码**：CompletionRecord.java + CompletionRecordRepository.java 是第一版使用的 Entity/Repository，第三版已替换为 ChatCompletionRecord + ChatCompletionRepository，但旧文件未删除。JacksonConfig.java 注册的 ObjectMapper Bean 与 Spring Boot 自动配置重复——冗余。
8. **测试客户端放在 src/main 而非 src/test**：OpenAiApiTest.java 和 OpenAiStreamTest.java 位于 src/main/java，会随生产 JAR 一起打包。正确做法是放在 src/test/java。
9. **Python SDK 测试仅覆盖非流式**：test_mock_api.py 只测试了 stream=False，未测试流式模式。考核文档要求同时验证流式和非流式。
10. **无 .gitignore**：项目缺少 .gitignore，target/ 等构建产物可能被误提交。

3 功能缺失

11. **仅一个单元测试**：contextLoads() 验证 Spring Context 启动，不测试任何业务逻辑。
12. **Model 参数不校验**：接受任意 model 字符串但不验证是否在可用模型列表中。
13. **H2 内存数据库**：数据重启即丢失。
14. **无任何可选加分项**：Files API、Admin API、API 中转、限流、并发控制均为空白。
15. **无 Swagger/Apifox 文档**：未提供 API 文档导出。

Suggestions / 修改建议

1. **扩大鉴权覆盖范围 (最高优先级)**：将 WebConfig 中 addPathPatterns("/v1/chat/completions") 改为 addPathPatterns("/v1/**")，使所有 /v1/ 路径均受鉴权保护
2. **修复 Cancel 路径**：将 @PostMapping("/v1/chat/completions/{id}/cancel") 改为 @PostMapping("/completions/{id}/cancel")
3. **实现真正的 Cancel**：将 SSE Thread 引用保存到 ConcurrentHashMap，cancel 时调用 thread.interrupt() 中断流式输出
4. **移除 Thread.sleep 阻塞**：从主请求线程移除 Thread.sleep(1000)；流式延迟应仅存在于 SSE 异步线程中

5. **清理注释代码和死代码**：移除 "First" / "Second" 注释版本、`CompletionRecord.java`、`CompletionRecordRepository.java`、`JacksonConfig.java`、`WebConfig.java` 中的旧版注释
6. **移动测试客户端**：将 `OpenAiApiTest.java` 和 `OpenAiStreamTest.java` 移至 `src/test/java`
7. **完善 Python SDK 测试**：添加 `stream=True` 的流式测试用例
8. **添加 .gitignore**：排除 `target/`、`.idea/`、`*.iml` 等

Questions for Authors / 给作者的问题

1. Cancel 功能中，你只设置了 `isCancelled=true` 字段并保存到数据库，但没有中断正在运行的 SSE 线程。你认为当前的 Cancel 实现是否真正起到了"取消正在处理的请求"的作用？如果流式响应已经开始发送给客户端，cancel 能否阻止剩余内容的发送？
2. `ChatController.chatCompletions()` 入口处有一个 `Thread.sleep(1000)`。你知道这行代码在非流式模式下会阻塞哪个线程吗？如果 10 个用户同时请求，会发生什么？

Overall Assessment / 总体评分

- **Core features:** 2.5/5.0 - Chat Completions（非流式 + SSE 流式）实现正确、Models API 可用、Bearer Token 鉴权代码实现正确、CRUD 持久化工作正常。**扣分原因**：鉴权仅覆盖 `/v1/chat/completions`（其余端点完全不受保护）、Cancel 端点路径错误（404）、Cancel 为伪实现（仅设标志不中断线程）、`Thread.sleep(1000)` 阻塞主线程
- **Extra features:** 0/2.0 - 无任何可选加分项实现（无 Files API、无 API 中转、无 Admin API、无限流）
- **Documentation completeness:** 1.0/2.0 - README 详尽、提供 Python SDK 测试脚本。扣分原因是缺少 Swagger/Apifox 文档导出、缺少 `.gitignore`
- **Code style:** 1.0/2.0 - 分层结构清晰、鉴权拦截器设计正确。扣分原因是 ~150 行注释旧代码、死代码未清理、测试客户端放在 `src/main`、Java raw type 警告（`Map<String, Object>` 强制转换）
- **Overall Assessment:** 4.5/10.0 - 项目展现了基本 Spring Boot 开发能力：拦截器、SSE 流式、JPA 持久化、Python SDK 测试等核心概念均有涉及。但完成度偏低——鉴权覆盖不完整使得项目的安全模型形同虚设（可通过无鉴权的 `/v1/models` 自由访问），Cancel 路径错误导致功能无法正常工作，主线程阻塞影响并发能力。若能修复鉴权覆盖范围、Cancel 路径和主线程阻塞三个关键问题，项目可提升到更完整的水平
- **Confidence:** 4.0/5.0 - 评审者熟悉 Java + Spring Boot 技术栈，已成功编译、启动和测试所有端点，验证了鉴权绕过、Cancel 路径错误等关键问题

面试参考问题（简单难度）

(wip)

刘品艺（基础扎实）

251732121 - 软件工程(大数据与人工智能学院)

第二轮考核

[简单记账web后端: 基于springboot+mybatis的简单记账web后端服务](#)

师弟or师妹你好，对于你的第二轮考核代码已经clone并且成功运行，由于时间紧，能完成到这个程度已经相当不错了，继续努力，对于你的代码项目有许多亮点，MVC结构清晰，接口遵循Restful风格，变量和类命名规范，还有记录日志的好习惯，接口文档清晰等等，可以看到你的学习痕迹，JWT，MyBatis等等，这么短的时间从学习到完成这个程度代表你的学习能力很强，同时也应该投入了很多时间，这里提出一些建议或许对你接下来的学习有帮助，可以去学习MyBatis-plus对于该项目使用这个框架，基本上可以把百分之90的Mapper代码省去，还有spring security，时间充足了需要学习，本项目利用JWT拦截也可以，代码结构中Mapper一块也可以利用接口实现，本项目的异常处理建议学习如何书写统一的异常处理，高级功能的实现跟普通的CRUD就完全不同了，可以尝试去书写高级功能，业务逻辑难度较高，期待接下来考核你的提交！

第三轮考核

Summary / 概述

本项目实现了一个 OpenAI API 兼容的多模型统一网关服务（Miroku-API），技术栈为 Java 21 + Spring Boot 4.0.5 + MyBatis + MySQL 8.0 + OkHttp + Sa-Token。项目核心亮点是**适配器模式（Strategy Pattern）**实现的多 LLM 提供商路由——通过抽象 `LlmAdapter` 基类，`DeepSeekAdapter` / `DoubaoAdapter` / `QwenAdapter` 三个实现类各自封装了与对应服务商的通信协议，新增模型仅需添加一个 Adapter 类即可零侵入扩展。

核心功能覆盖：Chat Completions（流式 SSE `StreamingResponseBody` + 非流式）、ID 改写（上游原始 ID → `Miroku-{UUID}` 保证生命周期一致性）、Completion CRUD（GET/DELETE）、Cancel（`OkHttp call.cancel()` + `ConcurrentHashMap` 追踪活跃调用）、Models API（配置文件驱动模型列表）、Bearer Token 鉴权（Sa-Token）。项目提供 Docker Compose 一键部署（MySQL 8.0 + 应用容器，健康检查），Flyway 自动建表迁移，Apifox 在线/离线 API 文档，以及基于官方 OpenAI Java SDK 的参数化集成测试（覆盖 4 个模

型 × 流式/非流式)。

经 Maven 编译验证：32 个源文件编译通过 (BUILD SUCCESS)，Spring Boot 4.0.5 依赖解析正常。

Strengths / 主要优点

- 适配器模式设计优雅：**LlmAdapter 抽象基类定义 support(model) + createChatCompletion(request) 两个核心方法，三个实现类各自封装 DeepSeek/豆包/千问的差异 (角色名映射、API 端点、鉴权头)。新增模型服务商零侵入——只需新增一个 Adapter 子类和对应的 @ConfigurationProperties 配置类。这是接口隔离原则和开闭原则的教科书级应用。
- 流式 SSE 实现正确且细节到位：**使用 Spring StreamingResponseBody 异步写入，逐行解析上游 SSE 流 (data: 行识别 → JSON 解析 → ID 替换 → 客户端转发)、正确处理 [DONE] 结束信号、\n\n 分隔符、每行 flush() 推送。Miroku-{UUID} ID 改写保证了跨服务商的生命周期一致性。流式过程中累积 fullContent 用于最终 DB 持久化。
- DB 持久化设计合理：**Flyway v1__init.sql 管理 MySQL 建表迁移 (response + choices 两张表，一对多关系)，MyBatis XML 手写 SQL (insertResponse / insertChoices / selectResponse / deleteResponse / deleteChoices)，异步 @Async("dbSaveExecutor") 保存避免阻塞主线程。取消删除时级联清理关联 choices。
- Cancel 实现为真取消：**与常见伪实现不同，本项目使用 ConcurrentHashMap<String, Call> 追踪活跃的 OkHttp Call 对象，cancel 时直接调用 call.cancel() 断开上游连接。Controller 层通过 tempCall.isCanceled() 判断取消并设置 finish_reason = "cancel"。
- 集成测试全面：**使用官方 OpenAI Java SDK (com.openai:openai-java:4.34.0) 编写外部集成测试，覆盖 4 个模型 (deepseek-chat / doubao-seed-2-0-mini / doubao-1-5-pro-32k / qwen3.6-flash)，每个模型均测试流式和非流式两种模式。测试通过 /login 动态获取 Token，验证 ID 前缀 Miroku、非空响应、chunk 计数等关键断言。
- Docker Compose 生产级配置：**镜像指定版本 (mysql:8.0、eclipse-temurin:21-jre-jammy)、健康检查 (MySQL mysqladmin ping + 应用 wget /login)、数据卷持久化、日志限制 (100MB/3 文件)、JVM 内存限制 (JAVA_OPTS 可覆盖)、网络隔离 (bridge)、优雅重启 (unless-stopped)。.env.example + .gitignore 保护敏感信息。
- API 文档完善：**Apifox 导出在线/离线 API 文档 (apifox/离线文档.md + apifox/在线文档.txt)，覆盖全部端点 (login / models / chat completions CRUD / cancel)，每个端点包含请求参数、响应示例、鉴权说明。
- 线程池隔离设计：**DbPool (10-20 线程, CallerRunsPolicy) 和 webPool (20-50 线程) 分离 DB 写入和 Web 请求的异步资源，避免 DB 慢写入阻塞 HTTP 响应。
- 错误处理分层：**GlobalExceptionHandler + @RestControllerAdvice 捕获 SaTokenException (→ 401) 和通用 Exception (→ 500)，返回 OpenAI 风格错误 JSON。

Weaknesses / 主要缺点

- 无单元测试：**唯一的测试 OpenAISDKExternalTest.java 是外部集成测试——需要 Docker 环境运行 MySQL + 应用 + 真实 LLM API Key。无 Mock 测试，无 Service/Adapter 层单元测试。mvn test 在非 Docker 环境下无法通过。
- CompletionController.create() 过长：**约 170 行的方法混合了 SSE 流解析、ID 改写、响应拼装、DB 保存等多个职责。建议拆分为 handleStreamResponse() / handleNonStreamResponse() / buildResponseFromSSELines() 等小方法。
- Login 为硬编码单塔菲：**LoginController 使用固定的 userId "Ace-Taffy-1883" 创建 Token，无真正的用户名/密码验证，无多用户支持。README 中诚实声明了这一局限性。—(塔菲厨 show show way)—
- 无 rate limiting / circuit breaker：**作为网关服务，缺乏对上游 LLM 服务商的调用频率限制和熔断保护。某个服务商故障时，请求会直接失败而非快速失败或回退到其他服务商。
- line.substring(0, 6) 存在越界风险：**CompletionController.java:77 对行长度小于 6 的 SSE 行直接做 substring 会抛出 StringIndexOutOfBoundsException。虽然正常 SSE 流不会产生这种行，但防御性编程应加长度检查。
- 非流式响应构建存在内存风险：**llmResponse.body().string() 将完整响应体读入内存字符串，对于超大响应 (如生成文) 可能造成内存压力。虽然实践中很少遇到，但可考虑流式读取 + 边读边写。

Suggestions / 修改建议

- 补充 Service 层单元测试：**使用 Mockito mock LlmAdapter 和 ResponseMapper，测试 CompletionServiceImpl.create() 的模型路由逻辑和 cancel 逻辑
- 拆分 CompletionController.create()：**将 170 行方法拆分为 handleStream() / handleNonStream() 等职责单一的小方法
- 修复 line.substring(0, 6) 越界风险：**加 if (line.length() < 6) 判断或使用 line.startsWith("data:") 替代
- 实现用户登录系统：**替换硬编码 userId，接入真实用户表 + 密码验证
- 考虑添加熔断降级：**使用 Resilience4j 或 Sentinel 对上游 LLM 调用做熔断保护，故障时快速失败或自动回退到备用服务商

Questions for Authors / 给作者的问题

(Not available)

Overall Assessment / 总体评分

- **Core features:** 4.5/5.0 - Chat Completions 流式 SSE（真正的 `StreamingResponseBody` 透传 + ID 改写）和非流式均正确实现、Models API 配置驱动、Bearer Token 鉴权有效、Completion CRUD 完整、Cancel 为真取消（OkHttp `call.cancel()`）。扣分原因：无单元测试（仅 Docker 依赖的外部集成测试）
- **Extra features:** 1.5/2.0 - 适配器模式实现多 LLM 服务商路由（DeepSeek + 豆包 + 千问）可视为 API 中转的高级实现、Docker Compose 生产级部署、Flyway 数据库迁移、Apifox API 文档、Cancel 真取消机制。扣分原因：无 rate limiting/circuit breaker、无 Admin API、Login 为硬编码单用户
- **Documentation completeness:** 1.5/2.0 - README 结构优良（架构图、鉴权流程、快速启动、SDK 测试、AIGC 声明）、Apifox 离线文档覆盖全端点、`.env.example` 提供模板。扣分原因：Apifox 文档中部分模型名与实际 `application.yml` 不完全一致、README 中标注 Spring Boot 3.x 但实际为 4.0.5
- **Code style:** 1.5/2.0 - 适配器模式设计优雅、Lombok 合理使用、DTO 分离清晰、线程池隔离、Flyway 规范建表。扣分原因：`CompletionController.create()` 170 行过长、`line.substring(0,6)` 越界风险、无 Service/Adapter 层单元测试、`pom.xml` 含模板残留空标签
- **Overall Assessment:** 9.0/10.0 - 项目展现了优秀的软件工程素养：适配器模式实现多模型路由、真正的 SSE 流式透传、生产级 Docker 配置、使用官方 OpenAI SDK 的参数化集成测试。在已评审的 Level 3 提交中，架构设计的优雅程度和流式实现的正确性均为前列。主要短板在于测试语言与考核文档要求不完全吻合、无单元测试、Login 为硬编码，以及网关层缺乏熔断限流等生产级防护
- **Confidence:** 4.0/5.0 - 评审者熟悉 Java + Spring Boot + MyBatis 技术栈，已成功编译全部 32 个源文件。对 OkHttp 流式处理、Sa-Token 鉴权、Flyway 迁移等相关技术有了解。未通过 Docker Compose 实际部署和运行集成测试（需真实 LLM API Key），因此集成测试的正确性基于代码审查判断

面试参考问题（中等难度）

(wip)

廖郁祺

第一轮考核

[AccountBookAir: 一款轻量的记账小程序](#)

整体评价

用 C++17 写了一个命令行记账软件，多用户注册登录、收支增删改查、分类统计、JSON 持久化这些基础功能都做完了，代码分了工具层/模型层/表现层三层，结构清晰。密码加盐哈希、金额用“分”存储避免浮点问题，这些细节说明你有基本的工程意识。作为初学者的第一个完整项目，完成度是 OK 的。

主要不足：进阶功能（预算管理、预设类型、管理员）一个没做；用户名密码允许空字符串注册；头文件里用了 `using std::string` 这类声明会污染命名空间；CMakeLists 引用了不存在的 `test/` 目录说明测试架子搭了但没写；README 的 macOS 依赖列表里有 `crow`、`mysql` 这些项目根本没用到的包，像是从别处抄来忘改了。

后续发展建议

1. **补一两个进阶功能：**预算管理或预设消费类型都不难，做完区分度会好很多。
2. **学会写单元测试：**这是你最明显的短板。从密码哈希验证开始，用 Google Test 或 Catch2 写几个测试用例，体会一下“改完代码跑一遍测试就知道有没有改坏东西”的感觉。
3. **输入校验是基本功：**任何有用户输入的程序都要做非空检查和长度限制，不只是记账软件。
4. **清理代码习惯：**删掉 `using namespace std;`，头文件里写全名 `std::string`；删掉 `login()` 末尾的死代码 `return nullptr;`；`findRecord()` 的两个重载应该复用 `findRecordIt()` 而不是各自重写一遍 `find_if`。
5. **完善 .gitignore：**加上 `*.o`、`*.obj`、`CMakeUserPresets.json`、`.vs/`、`.idea/`、`data/`，避免把编译产物和本地配置提交上去。
6. **理解 AI 生成的代码：**JSON 持久化和密码哈希是你用 DeepSeek 生成的，面试时会被问到——salt 怎么生成的、为什么用 SHA-256 不用 MD5、JSON 序列化怎么处理嵌套对象。确保你能解释每一行。

第二轮考核

<https://gitee.com/Mirai-39/AccountBook>

学弟学妹你好，看了你的代码，能感觉到你是认真做了的。用 C++17 + Crow + MySQL 搭了一套完整的 REST API，三层架构分得挺清楚——路由层、业务逻辑层、数据访问层各管各的，中间件鉴权和管理员权限也做了，参数化 SQL 查询没有注入风险，密码加了盐哈希，金额用“分”存储避免浮点问题，文档也写得比较全（数据库设计、部署指南、Apifox 文档都有）。从 Level 1 的命令行记账本升级到 Level 2 的 HTTP API，这个进步是实实在在的，说明你确实是在学东西。

不过有几个比较关键的问题需要正视。最核心的一个是：你虽然接了 MySQL，但启动时把所有用户和所有账单全部加载到内存里，之后所有的查询、统计、分页都是在内存里做的，数据库只是当了个"持久化存储"用——这意味着数据量一大内存就炸了，而且分页和筛选完全没有下推到 SQL 层，等于白用了数据库。第二个是服务器只开了一个线程（`concurrency(1)`），任何请求慢了整个服务就卡住了。安全方面，token 永远不过期、SHA-256 没有做 key stretching（应该用 bcrypt 或 argon2）、管理员可以把自己的账号禁用掉、`std::stoi` 解析分页参数没做异常处理——这些都是面试时大概率会被问到的点。代码层面也有一些小问题：`changeUserPassword` 里密码存了两次（`User::changePassword` 存一次，`UserManager` 又存一次）、日期格式化不统一（有的补零有的不补）、测试代码编译进了生产二进制、`reinterpret_cast` 传 `userId` 的写法比较脆弱。

整体来说，你已经具备了用 C++ 写一个能跑的后端服务的能力，这在初学者里是不错的起点。希望你能从这几个方向继续提升：一是学会让数据库真正干活——把查询、过滤、分页都放到 SQL 里做，不要全拉到内存；二是关注安全细节——token 加个过期时间、密码哈希换 bcrypt、输入参数做校验；三是把测试从 `main()` 里解放出来，用 Google Test 之类的框架独立跑，这样改代码才有底气。如果能进入面试环节的话，建议你重点准备"为什么选择把数据全部加载到内存"和"token 为什么没有过期机制"这两个问题，想清楚当时的取舍和改进方案。祝你学业有成，继续加油。

面试参考问题（简单难度）

(wip)